



React Composition and Hooks

Advanced Patterns for Scalable Applications

Course Overview

- 01 Custom Hooks
- 02 Context API
- 03 useReducer
- 04 Component Composition
- 05 React Patterns
- 06 Testing & Production
- 07 Beyond React



Custom Hooks

Section 1



Why Custom Hooks?

The Problem

- Components often share the same stateful logic
- Copy-pasting `useState/useEffect` code between components

The Solution

- Extract shared logic into a reusable function
- Custom hooks are functions that start with 'use'
- They can call other hooks — `useState`, `useEffect`, etc.



Custom Hooks — Reusable Logic

Custom Hooks — Reusable Logic

✗ Before: Duplicated Logic

```
function UserList() {  
  // fetch logic here  
  const [data, setData] = ...  
  const [loading, setLoading] = ...  
  useEffect(() => fetch(...))  
}
```

```
function PostList() {  
  // same fetch logic again!  
  const [data, setData] = ...  
  const [loading, setLoading] = ...  
  useEffect(() => fetch(...))  
}
```

✓ After: Custom Hook

```
function useFetch<T>(url) {  
  const [data, setData] = ...  
  const [loading, setLoading] = ...  
  useEffect(() => fetch(...))  
  return { data, loading }  
}
```

```
function UserList() {  
  const { data } = useFetch('/users')  
}
```

```
function PostList() {  
  const { data } = useFetch('/posts')  
}
```

Rule: Custom hook names must start with use

Can use: useState, useEffect, useContext, other hooks

Returns: Any value — object, array, tuple



Building a useFetch Hook

```
function useFetch<T>(url: string) {
  const [data, setData] = useState<T | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    const controller = new AbortController();
    fetch(url, { signal: controller.signal })
      .then(res => {
        if (!res.ok) throw new Error(res.statusText);
        return res.json();
      })
      .then(setData)
      .catch(err => {
        if (err.name === 'AbortError') return;
        setError(err.message);
      })
      .finally(() => setLoading(false));
    return () => controller.abort();
  }, [url]);

  return { data, loading, error };
}
```



Using Custom Hooks

```
// Any component can now use the shared logic
function UserList() {
  const { data, loading, error } = useFetch<User[]>('/api/users');

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <ul>
      {data?.map(u => <li key={u.id}>{u.name}</li>)}
    </ul>
  );
}
```



Custom Hook Patterns

Common Custom Hooks

- `useLocalStorage` — persist state to `localStorage`
- `useDebounce` — delay value updates for search inputs
- `useMediaQuery` — respond to screen size changes
- `useToggle` — boolean state with toggle function

Rules

- Must start with 'use' — React enforces hook rules
- Each component gets its own copy of the hook's state



Context API

Section 2



The Prop Drilling Problem

What Is Prop Drilling?

- Passing props through intermediate components that don't use them
- App → Layout → Sidebar → Button (all pass 'theme')

Why It's a Problem

- Intermediate components become coupled to data they don't need
- Adding a new prop requires changes across the entire chain
- Makes refactoring difficult and error-prone



React Context API

React Context API

✗ Prop Drilling

App `theme=dark`

↓ props

Layout `theme=dark`

↓ props

Sidebar `theme=dark`

↓ props

Button `theme=dark`

Every intermediate component passes theme, even if it doesn't use it

✓ Context API

`ThemeContext.Provider value={ theme }`

Layout

Sidebar

Button
`useContext()`

Button reads theme directly — no drilling through intermediate components

1. Create

```
const ThemeCtx = createContext('light')
```

2. Provide

```
<ThemeCtx.Provider value='dark'>
```

3. Consume

```
const theme = useContext(ThemeCtx)
```



Creating and Providing Context

```
// 1. Create the context with a typed default
type ThemeContextType = { theme: 'light' | 'dark'; setTheme: (t: 'light' | 'dark') => void };
const ThemeContext = createContext<ThemeContextType>(
  { theme: 'light', setTheme: () => {} }
);

// 2. Create a provider component
function ThemeProvider({ children }: { children: ReactNode }) {
  const [theme, setTheme] = useState<'light' | 'dark'>('light');

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}
```



Consuming Context with useContext

```
// 3. Consume context – no prop drilling!
function Button() {
  const { theme, setTheme } = useContext(ThemeContext);

  return (
    <button
      className={theme === 'dark' ? 'btn-dark' : 'btn-light'}
      onClick={() => setTheme(theme === 'dark' ? 'light' : 'dark')}
    >
      Toggle Theme
    </button>
  );
}

// Custom hook for cleaner access
function useTheme() { return useContext(ThemeContext); }
```



Context Best Practices

When to Use Context

- Theme, locale, authentication — truly global state
- Data needed by many components at different nesting levels

When NOT to Use Context

- Data only needed by one or two components — just pass props
- Frequent updates are allowed but may cause re-renders unless optimized (e.g., split contexts or memoize)
- Tip: Create a custom hook (useTheme, useAuth) to wrap useContext



useReducer

Section 3



When useState Isn't Enough

useState Limitations

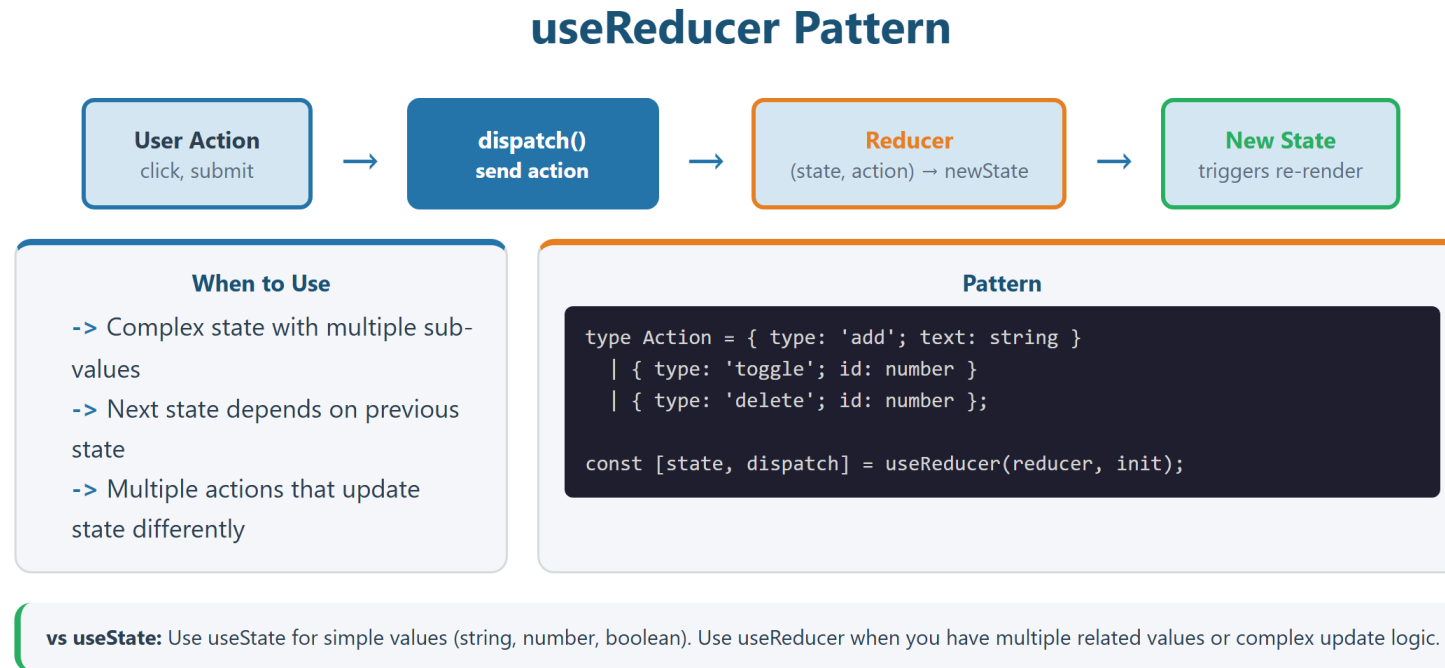
- Multiple related state values that change together
- Complex update logic scattered across event handlers

useReducer Advantages

- Centralizes all state transitions in one function
- Actions describe what happened, reducer decides how state changes
- Easy to test — pure function with predictable output



useReducer Pattern



useReducer enables predictable state transitions — testable, centralized, and scalable



Defining Actions and Reducer

```
type Todo = { id: number; text: string; done: boolean };

type Action =
  | { type: 'add'; text: string }
  | { type: 'toggle'; id: number }
  | { type: 'delete'; id: number };

function todoReducer(state: Todo[], action: Action): Todo[] {
  switch (action.type) {
    case 'add':
      return [...state, { id: Date.now(), text: action.text, done: false }];
    case 'toggle':
      return state.map(t => t.id === action.id ? { ...t, done: !t.done } : t);
    case 'delete':
      return state.filter(t => t.id !== action.id);
    default:
      return state;
  }
}
```



Using useReducer in a Component

```
function TodoApp() {
  const [todos, dispatch] = useReducer(todoReducer, []);

  return (
    <div>
      <AddTodo onAdd={text => dispatch({ type: 'add', text })} />
      {todos.map(t => (
        <TodoItem
          key={t.id}
          todo={t}
          onToggle={() => dispatch({ type: 'toggle', id: t.id })}
          onDelete={() => dispatch({ type: 'delete', id: t.id })}
        />
      ))}
    </div>
  );
}
```



useReducer + Context

Powerful Combination

- useReducer manages complex state logic
- Context distributes state and dispatch to the component tree

Pattern

- Create a provider that wraps useReducer
- Provide both state and dispatch via context
- Any nested component can read state or dispatch actions
- Suitable for app-local and shared state — not a full replacement for external libraries like Zustand or Redux



Component Composition

Section 4



The children Prop

What Is children?

- Content placed between a component's opening and closing tags
- The most basic composition pattern in React

Why It Matters

- Components don't need to know their content in advance
- A Card component can wrap any content — text, images, forms
- Keeps components generic and reusable



Composition with children

```
// Generic container – doesn't know its content
function Card({ title, children }: {
  title: string;
  children: ReactNode;
}) {
  return (
    <div className="card">
      <h2>{title}</h2>
      <div className="card-body">{children}</div>
    </div>
  );
}

// Usage – any content works
<Card title="Profile"><UserAvatar /><UserBio /></Card>
<Card title="Settings"><SettingsForm /></Card>
```



Render Props Pattern

What Are Render Props?

- A prop whose value is a function that returns JSX
- The component calls the function, passing data as arguments

Example Use Cases

- Mouse position tracker — renders whatever you want at cursor location
- Data provider — fetches data, lets consumer decide how to render
- Note: custom hooks have largely replaced render props for logic sharing



Render Props Example

```
// Component that provides data via a render function
function DataFetcher<T>({ url, render }: {
  url: string;
  render: (data: T | null, loading: boolean) => ReactNode;
}) {
  const { data, loading } = useFetch<T>(url);
  return <>{render(data, loading)}</>;
}

// Consumer decides how to display the data
<DataFetcher<User[]>
  url="/api/users"
  render={users =>
    loading ? <Spinner /> : <UserTable users={users} />
  }
/>
```



Higher-Order Components

What Is an HOC?

- A function that takes a component and returns a new enhanced component
- Pattern: `withFeature(WrappedComponent) → EnhancedComponent`

Common Examples

- `withAuth` — wraps a component with authentication checks
- `withTheme` — injects theme props into a component

Modern Alternative

- Custom hooks are now preferred for most HOC use cases
- HOCs are still common in existing codebases and libraries



React Patterns

Section 5



Controlled vs Uncontrolled Components

Controlled Components

- React state is the single source of truth for form values
- Every change goes through `setState` → re-render
- Full control over validation, formatting, and submission

Uncontrolled Components

- DOM itself holds the form state — accessed via `useRef`
- Simpler for basic forms where you only need values on submit
- Both are valid — uncontrolled inputs work well for simple forms and third-party integrations



Controlled vs Uncontrolled

```
// CONTROLLED – React manages the value
function ControlledInput() {
  const [value, setValue] = useState('');
  return <input value={value} onChange={e => setValue(e.target.value)} />;
}

// UNCONTROLLED – DOM manages the value
function UncontrolledInput() {
  const inputRef = useRef<HTMLInputElement>(null);
  const handleSubmit = () => alert(inputRef.current?.value);

  return (
    <form onSubmit={e => { e.preventDefault(); handleSubmit(); }}>
      <input ref={inputRef} defaultValue="" />
      <button type="submit">Submit</button>
    </form>
  );
}
```



Compound Components Pattern

What Are Compound Components?

- A set of components that work together to form a complete UI
- Parent manages shared state, children render parts of the UI

Examples in the Wild

- `<Select>` + `<Option>` — HTML's built-in compound component
- `<Tabs>` + `<Tab>` + `<TabPanel>` — common UI library pattern
- `<Accordion>` + `<AccordionItem>` — expandable sections



Compound Component Example

```
// Parent manages state, shares via context
const TabsContext = createContext<{
  active: string;
  setActive: React.Dispatch<React.SetStateAction<string>>;
} | null>(null);

type Props = { children: ReactNode; defaultTab: string };

function Tabs({ children, defaultTab }: Props) {
  const [active, setActive] = useState(defaultTab);
  return (
    <TabsContext.Provider value={{ active, setActive }}>
      <div className="tabs">{children}</div>
    </TabsContext.Provider>
  );
}

// Usage – clean, declarative API
<Tabs defaultTab="overview">
  <Tab id="overview">Overview</Tab>
  <Tab id="details">Details</Tab>
  <TabPanel id="overview"><Overview /></TabPanel>
</Tabs>
```



React.memo and Performance

Why Components Re-render

- A component re-renders when its parent re-renders
- Even if the component's props haven't changed (this is the default behavior)

React.memo

- Wraps a component to skip re-renders if props are the same
- Uses shallow comparison by default

useMemo and useCallback

- useMemo — caches expensive computed values
- useCallback — caches function references to prevent child re-renders



Performance Optimization

```
import { memo, useMemo, useCallback } from 'react';

// React.memo – skip re-render if props unchanged
const ExpensiveList = memo(function ExpensiveList({ items }: Props) {
  return items.map(item => <Item key={item.id} {...item} />);
});

// useMemo – cache expensive computation
const sorted = useMemo(
  () => [...items].sort((a, b) => a.name.localeCompare(b.name)),
  [items]
);

// useCallback – stable function reference
// Note: dispatch from useReducer is stable; no need in deps
const handleClick = useCallback(
  (id: number) => dispatch({ type: 'toggle', id }),
  [] // dispatch is stable – safe to omit
);
```



Testing and Production

Section 6



Testing React Components

React Testing Library Philosophy

- Test components the way users interact with them
- Prefer accessible queries (role, label, text); data-testid is still valid when no accessible query fits

Key Utilities

- `render()` — renders a component into a virtual DOM
- `screen.getByRole()` — finds elements by their accessible role
- `fireEvent` / `userEvent` — simulates user interactions
- `waitFor()` — waits for async updates to complete



Writing a Component Test

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { Counter } from './Counter';

test('increments count on click', async () => {
  render(<Counter />);

  const button = screen.getByRole('button', { name: /increment/i });
  expect(screen.getByText('Count: 0')).toBeInTheDocument();

  await userEvent.click(button);
  expect(screen.getByText('Count: 1')).toBeInTheDocument();
});
```



Testing Custom Hooks

Preferred Approach

- Test hooks indirectly by testing the components that use them
- This tests real behavior and avoids testing implementation details

renderHook — For Hook Libraries

- `renderHook(() => useMyHook())` from `@testing-library/react`
- Best suited for reusable hook libraries or hooks with complex return values
- Use `act()` to trigger state updates inside the hook



Testing a Custom Hook

```
import { renderHook, act } from '@testing-library/react';
import { useCounter } from './useCounter';

// Direct hook testing – appropriate for reusable hook
// libraries or hooks not easily tested via a component
test('useCounter increments and resets', () => {
  const { result } = renderHook(() => useCounter(0));

  expect(result.current.count).toBe(0);

  act(() => result.current.increment());
  expect(result.current.count).toBe(1);

  act(() => result.current.reset());
  expect(result.current.count).toBe(0);
});
```



Testing Components with Context

The Problem

- Components using useContext fail without a Provider
- Tests must wrap the component in the required Provider

Custom Render Helper

- Create a renderWithProviders() wrapper for your tests
- Pass initial state as an option to control test scenarios

Testing Dispatched Actions

- Render with a known state, interact with the UI, assert the result
- Verify that user actions produce the expected state changes



Testing a Context Consumer

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { TodoProvider } from './TodoContext';
import { TodoList } from './TodoList';

function renderWithProvider(ui: React.ReactElement) {
  return render(<TodoProvider>{ui}</TodoProvider>);
}

test('adds a new todo item', async () => {
  renderWithProvider(<TodoList />);

  await userEvent.type(screen.getByRole('textbox'), 'Buy milk');
  await userEvent.click(screen.getByRole('button', { name: /add/i }));

  expect(screen.getByText('Buy milk')).toBeInTheDocument();
});
```

Testing Async and Integration Patterns

Async Testing

- Use `findByRole` / `findByText` for elements that appear after async operations
- `waitFor()` retries assertions until they pass or time out
- Mock fetch with `vi.fn()` or `msw` (Mock Service Worker) for API tests

Integration Testing Tips

- Test the full component tree — parents, children, and context together
- Focus on user workflows: fill form → submit → see result
- Avoid testing implementation details (internal state, method calls)



Preparing for Production

Build Process

- `npm run build` — creates optimized production bundle (Vite, CRA, or your bundler)
- Tree-shaking removes unused code automatically
- Assets are minified, hashed, and split into chunks

Pre-Deployment Checklist

- Remove `console.log` statements and debugging code
- Set environment variables for production API URLs
- Run the full test suite — `npm test`
- Test the production build locally — `npm run preview` (Vite-specific)



Build and Deploy Commands

```
# Build for production
npm run build

# Preview the production build locally
npm run preview

# Output in dist/ directory:
#   dist/
#   ├── index.html
#   ├── assets/
#   │   ├── index-abc123.js    (minified bundle)
#   │   └── index-def456.css  (minified styles)

# Deploy dist/ to any static host:
#   Netlify, Vercel, GitHub Pages, S3 + CloudFront

# Note: apps using SSR (Next.js, Remix) require
# a Node.js server or serverless deployment instead
```



Beyond React

Section 7



The Frontend Landscape

React Is Not the Only Way

- React is one popular choice — the industry has many strong alternatives
- The patterns you've learned (components, state, composition) transfer across frameworks

Other Modern Frameworks

- Vue — progressive framework with template syntax and built-in reactivity
- Svelte — compiles components at build time, minimizing framework runtime overhead
- Angular — full-featured platform with built-in DI, RxJS, and emerging signals-based reactivity
- SolidJS — fine-grained reactivity with React-like syntax
- Qwik — resumable apps with instant startup and minimal hydration
- Astro — server-first framework with island-based hydration



The Modern Frontend Landscape

The Modern Frontend Landscape

React

- > Library — UI layer only
- > JSX + component model
- > Virtual DOM diffing
- > Massive ecosystem
- > **Used in this program**

Vue

- > Progressive framework
- > Template syntax + SFCs
- > Reactive data binding
- > Official router & state mgmt
- > Gentle learning curve

Svelte

- > Compiler — no runtime
- > Minimal boilerplate
- > Reactive by assignment
- > Small bundle size
- > SvelteKit for full-stack

Next.js (React meta-framework)

- > Server-side rendering & static generation
- > File-based routing
- > API routes built-in
- > React Server Components
- > **Used at Liberty for newer apps**

Key Takeaway

React is **one excellent choice** among many. The core concepts you've learned — components, state management, and composition — transfer directly to Vue, Svelte, and other frameworks. What matters most is understanding the *patterns*, not just one tool.

Next.js and Meta-Frameworks

What Are Meta-Frameworks?

- Meta-frameworks extend UI libraries with routing, server rendering, data fetching, and full-stack capabilities (APIs, edge, streaming)

Next.js (React)

- Supports React Server Components (RSC) for server-first rendering
- App Router enables layouts, streaming, and nested routing
- Hybrid rendering: SSR, SSG, ISR, and edge rendering
- Built-in data fetching, caching, and server actions



Key Takeaways

Key Takeaways

1. Custom hooks extract and share stateful logic between components
2. Context API eliminates prop drilling for global state like themes and auth
3. `useReducer` centralizes complex state transitions in a testable reducer function
4. Composition patterns (children, render props) keep components flexible
5. `React.memo`, `useMemo`, and `useCallback` optimize rendering performance
6. React Testing Library tests components the way users interact with them
7. Vue and Svelte are alternatives to React. Next.js is a React-based meta-framework — your React patterns transfer

